

SLR Parser Construction



- 과목명: 컴파일러
- 담당교수님: 문희승 교수님
- 학과: 소프트웨어학부
- 그룹: 1032
- 보고서작성자: 20220702 서정민

[목차]

1. Grammar Refactoring

1-1. dangling else 문제

1-2. 연산자 우선순위와 결합 순서 문제

2. SLR(1) Table Construction

2-1. Table Construction

2-2. Follow 집합이 테이블의 충돌을 해결하는 예시

3. Parser Implementation

3-1. 구현

3-2. 결과 예시

1. Grammar Refactoring

주어진 문법에서 문법이 모호한 문제를 먼저 해결한다.

1-1. 11, 12번의 dangling else 문제

11. Stmt -> if (Expr) Stmt

12. Stmt -> if (Expr) Stmt else Stmt

이 문법은 모호하다. 11번에서 18번까지의 문법을 아래와 같이 수정한다.

Stmt -> IfStmt | OtherStmt

IfStmt -> matched_Stmt | open_Stmt

matched_Stmt -> if (Expr) then matched_Stmt else matched_Stmt | OtherStmt

open_Stmt -> if (Expr) then Stmt | if (Expr) then matched_Stmt else open_Stmt

OtherStmt -> while (Expr) Stmt | for (Expr ; Expr ; Expr) Stmt | return Expr; | VarDecl | ExprStmt
| Block

1-2. 20번에서 27번까지의 연산자 우선순위와 결합 순서 문제

20. $\text{Expr} \rightarrow \text{Expr} == \text{Expr}$

21. $\text{Expr} \rightarrow \text{Expr} + \text{Expr}$

22. $\text{Expr} \rightarrow \text{Expr} * \text{Expr}$

이 문법은 모호하다. 연산자 우선순위 문제가 있고, 그 아래도 우선순위 문제로 수정해야 한다.

$\text{Expr} \rightarrow \text{EqualExpr}$

$\text{EqualExpr} \rightarrow \text{EqualExpr} == \text{AddExpr} \mid \text{AddExpr}$

$\text{AddExpr} \rightarrow \text{AddExpr} + \text{MulExpr} \mid \text{MulExpr}$

$\text{MulExpr} \rightarrow \text{MulExpr} * \text{UnaryExpr} \mid \text{UnaryExpr}$

$\text{UnaryExpr} \rightarrow - \text{UnaryExpr} \mid \text{OtherExpr}$

$\text{OtherExpr} \rightarrow \text{id} (\text{ArgList}) \mid \text{id} \mid \text{num} \mid (\text{Expr})$

수정한 전체 문법은 아래와 같다.

1. $\text{Program} \rightarrow \text{DeclList}$
2. $\text{DeclList} \rightarrow \text{Decl DeclList} \mid \text{epsilon}$
3. $\text{Decl} \rightarrow \text{VarDecl} \mid \text{FuncDecl}$
4. $\text{VarDecl} \rightarrow \text{type id} ; \mid \text{type id} = \text{Expr} ;$
5. $\text{FuncDecl} \rightarrow \text{type id} (\text{ParamList}) \text{Block}$
6. $\text{ParamList} \rightarrow \text{Param}, \text{ParamList} \mid \text{Param} \mid \text{epsilon}$
7. $\text{Param} \rightarrow \text{type id}$
8. $\text{Block} \rightarrow \{ \text{StmtList} \}$
9. $\text{StmtList} \rightarrow \text{Stmt StmtList} \mid \text{epsilon}$
10. $\text{Stmt} \rightarrow \text{IfStmt} \mid \text{OtherStmt}$
11. $\text{IfStmt} \rightarrow \text{matched_Stmt} \mid \text{open_Stmt}$
12. $\text{matched_Stmt} \rightarrow \text{if} (\text{Expr}) \text{ then matched_Stmt else matched_Stmt} \mid \text{OtherStmt}$
13. $\text{open_Stmt} \rightarrow \text{if} (\text{Expr}) \text{ then Stmt} \mid \text{if} (\text{Expr}) \text{ then matched_Stmt else open_Stmt}$
14. $\text{OtherStmt} \rightarrow \text{while} (\text{Expr}) \text{ Stmt} \mid \text{for} (\text{Expr} ; \text{Expr} ; \text{Expr}) \text{ Stmt} \mid \text{return Expr}; \mid \text{VarDecl} \mid \text{ExprStmt} \mid \text{Block}$

- 15. ExprStmt -> id = Expr ;
- 16. Expr -> EqualExpr
- 17. EqualExpr -> EqualExpr == AddExpr | AddExpr
- 18. AddExpr -> AddExpr + MulExpr | MulExpr
- 19. MulExpr -> MulExpr * UnaryExpr | UnaryExpr
- 20. UnaryExpr -> - UnaryExpr | OtherExpr
- 21. OtherExpr -> id (ArgList) | id | num | (Expr)
- 22. ArgList -> Expr, ArgList | Expr | epsilon

두번째로 reduce/reduce 충돌을 해결한다.

Program -> DeclList

DeclList -> Decl DeclList

DeclList -> epsilon

Decl -> VarDecl

Decl -> FuncDecl

VarDecl -> type id ;

VarDecl -> type id = Expr ;

FuncDecl -> type id (ParamList) Block

ParamList -> Param , ParamList

ParamList -> Param

ParamList -> epsilon

Param -> type id

Block -> { StmtList }

StmtList -> Stmt StmtList

StmtList -> epsilon

Stmt -> IfStmt

Stmt -> OtherStmt

IfStmt -> matched_Stmt

IfStmt -> open_Stmt

matched_Stmt -> if (Expr) then matched_Stmt else matched_Stmt

matched_Stmt -> OtherStmt

open_Stmt -> if (Expr) then Stmt

open_Stmt -> if (Expr) then matched_Stmt else open_Stmt

OtherStmt -> while (Expr) Stmt

OtherStmt -> for (Expr ; Expr ; Expr) Stmt

OtherStmt -> return Expr ;

OtherStmt -> VarDecl

OtherStmt -> ExprStmt

OtherStmt -> Block

ExprStmt -> id = Expr ;

Expr -> EqualExpr

EqualExpr -> EqualExpr == AddExpr

EqualExpr -> AddExpr

AddExpr -> AddExpr + MulExpr

AddExpr -> MulExpr

MulExpr -> MulExpr * UnaryExpr

MulExpr -> UnaryExpr

UnaryExpr -> - UnaryExpr

UnaryExpr -> OtherExpr

OtherExpr -> id (ArgList)

OtherExpr -> id

OtherExpr -> num

OtherExpr -> (Expr)

ArgList -> Expr , ArgList

ArgList -> Expr

ArgList -> epsilon

이 문법을 이용해서 SLR Table Construction을 진행한다.

<https://jsmachines.sourceforge.net/machines/slr.html> 과제에서 주어진 이 사이트를 이용한다. 위의 문법으로 바로 Table Construction을 해보면 충돌이 발생하는 것을 확인할 수 있다.

위 문법에서 구한 FIRST, FOLLOW set

FIRST / FOLLOW table		
Nonterminal	FIRST	FOLLOW
Program	{epsilon,type}	{\$}
DeclList	{epsilon,type}	{\$}
Decl	{type}	{epsilon,type}
VarDecl	{type}	{epsilon,type,while,for,return,{,if,id,else}
FuncDecl	{type}	{epsilon,type}
ParamList	{epsilon,type}	{}
Param	{type}	{,,}
Block	{}	{epsilon,type,while,for,return,{,if,id,else}
StmtList	{epsilon,while,for,return,type,{,if,id}	{}
Stmt	{while,for,return,type,{,if,id}	{epsilon,while,for,return,type,{,if,id,else}
IfStmt	{if,while,for,return,type,{,id}	{epsilon,while,for,return,type,{,if,id,else}
matched_Stmt	{if,while,for,return,type,{,id}	{epsilon,while,for,return,type,{,if,id,else}
open_Stmt	{if}	{epsilon,while,for,return,type,{,if,id,else}
OtherStmt	{while,for,return,type,{,id}	{epsilon,while,for,return,type,{,if,id,else}
ExprStmt	{id}	{epsilon,while,for,return,type,{,if,id,else}
Expr	{AddExp}	{;,},,,}
EqualExpr	{AddExp}	{;,},==,,}
AddExpr	{UnaryExp}	{;,},==,+,,,}
MulExpr	{UnaryExp}	{;,},==,+,*,,,}
UnaryExpr	{-,id,num,() }	{;,},==,+,*,,,}
OtherExpr	{id,num,() }	{;,},==,+,*,,,}
ArgList	{epsilon,AddExp}	{}

위 문법에서 구한 LR Table

The image shows a large, complex LR Table for a grammar. The table is organized into sections for different parts of the grammar, such as declarations, statements, and expressions. The table is filled with entries for transitions between states, including shift, reduce, and goto actions. The table is organized into sections for different parts of the grammar, such as declarations, statements, and expressions. The table is filled with entries for transitions between states, including shift, reduce, and goto actions.

먼저, 위부분에서 Reduce/reduce 충돌이 발생하고 있다. 이것은 위의 수정한 문법에서 10번이 불필요하기 때문이다.

10. Stmt -> IfStmt | OtherStmt

Stmt -> OtherStmt

Matched_Stmt -> OtherStmt

두개의 축약 규칙이 동시에 적용되기에 발생하는 문제이다.

다시 문법을 수정한다.

1. Program -> DeclList
2. DeclList -> Decl DeclList | epsilon
3. Decl -> VarDecl | FuncDecl

4. VarDecl -> type id ; | type id = Expr ;
5. FuncDecl -> type id (ParamList) Block
6. ParamList -> Param, ParamList | Param | epsilon
7. Param -> type id
8. Block -> { StmtList }
9. StmtList -> Stmt StmtList | epsilon
10. Stmt -> matched_Stmt | open_Stmt
11. matched_Stmt -> if (Expr) then matched_Stmt else matched_Stmt | OtherStmt
12. open_Stmt -> if (Expr) then Stmt | if (Expr) then matched_Stmt else open_Stmt
13. OtherStmt -> while (Expr) Stmt | for (Expr ; Expr ; Expr) Stmt | return Expr; | VarDecl | ExprStmt | Block
14. ExprStmt -> id = Expr ;
15. Expr -> EqualExpr
16. EqualExpr -> EqualExpr == AddExpr | AddExpr
17. AddExpr -> AddExpr + MulExpr | MulExpr
18. MulExpr -> MulExpr * UnaryExpr | UnaryExpr
19. UnaryExpr -> - UnaryExpr | OtherExpr
20. OtherExpr -> id (ArgList) | id | num | (Expr)
21. ArgList -> Expr, ArgList | Expr | epsilon

OtherStmt가 한 곳에서만 축약되도록 문법을 수정 후 다시 Table Construction한다.

Program -> DeclList

DeclList -> Decl DeclList

DeclList -> epsilon

Decl -> VarDecl

Decl -> FuncDecl

VarDecl -> type id ;

VarDecl -> type id = Expr ;

FuncDecl -> type id (ParamList) Block

ParamList -> Param , ParamList

ParamList -> Param

ParamList -> epsilon

Param -> type id

Block -> { StmtList }

StmtList -> Stmt StmtList

StmtList -> epsilon

Stmt -> matched_Stmt

Stmt -> open_Stmt

matched_Stmt -> if (Expr) then matched_Stmt else matched_Stmt

matched_Stmt -> OtherStmt

open_Stmt -> if (Expr) then Stmt

open_Stmt -> if (Expr) then matched_Stmt else open_Stmt

OtherStmt -> while (Expr) Stmt

OtherStmt -> for (Expr ; Expr ; Expr) Stmt

OtherStmt -> return Expr ;

OtherStmt -> VarDecl

OtherStmt -> ExprStmt

OtherStmt -> Block

ExprStmt -> id = Expr ;

Expr -> EqualExpr

EqualExpr -> EqualExpr == AddExpr

EqualExpr -> AddExpr

AddExpr -> AddExpr + MulExpr

AddExpr -> MulExpr

MulExpr -> MulExpr * UnaryExpr

MulExpr -> UnaryExpr

UnaryExpr -> - UnaryExpr

UnaryExpr -> OtherExpr

OtherExpr -> id (ArgList)

OtherExpr -> id

OtherExpr -> num

OtherExpr -> (Expr)

ArgList -> Expr , ArgList

ArgList -> Expr

ArgList -> epsilon

reduce/reduce 충돌이 해결되었다. 그러나 아래 shift/reduce 충돌이 있다.

IfStmt 에서 ElseStmt 부분을 분리하고 if 문 안에서 Stmt 자리에 Block을 사용해서 끝을 명확하게 하도록 만들었다. 그리고 SLR(1) 테이블 생성 이전에 Program' -> Program 부분을 추가하고 테이블을 생성한다.

Program' -> Program

Program -> DeclList

DeclList -> Decl DeclList

DeclList -> epsilon

Decl -> VarDecl

Decl -> FuncDecl

VarDecl -> type id ;

VarDecl -> type id = Expr ;

FuncDecl -> type id (ParamList) Block

ParamList -> Param , ParamList

ParamList -> Param

ParamList -> epsilon

Param -> type id

Block -> { StmtList }

StmtList -> Stmt StmtList

StmtList -> epsilon

Stmt -> IfStmt

Stmt -> OtherStmt

IfStmt -> if (Expr) then Block ElseStmt

ElseStmt -> else Block

ElseStmt -> epsilon

OtherStmt -> while (Expr) Stmt

OtherStmt -> for (Expr ; Expr ; Expr) Stmt

OtherStmt -> return Expr ;

OtherStmt -> VarDecl

OtherStmt -> ExprStmt

OtherStmt -> Block

ExprStmt -> id = Expr ;

Expr -> EqualExpr

EqualExpr -> EqualExpr == AddExpr

EqualExpr -> AddExpr

AddExpr -> AddExpr + MulExpr

AddExpr -> MulExpr

MulExpr -> MulExpr * UnaryExpr

MulExpr -> UnaryExpr

UnaryExpr -> - UnaryExpr

UnaryExpr -> OtherExpr

OtherExpr -> id (ArgList)

OtherExpr -> id

OtherExpr -> num

OtherExpr -> (Expr)

ArgList -> Expr , ArgList

ArgList -> Expr

ArgList -> epsilon

2. SLR Table Construction

2-1. Table Construction

FIRST / FOLLOW table		
Nonterminal	FIRST	FOLLOW
Program'	{epsilon,type}	{\$}
Program	{epsilon,type}	{\$}
DeclList	{epsilon,type}	{\$}
Decl	{type}	{epsilon,type}
VarDecl	{type}	{epsilon,type,if,while,for,return,{,id}
FuncDecl	{type}	{epsilon,type}
ParamList	{epsilon,type}	{}
Param	{type}	{,,}
Block	{{}	{epsilon,type,else,if,while,for,return,{,id}
StmtList	{epsilon,if,while,for,return,type,{,id}	{}
Stmt	{if,while,for,return,type,{,id}	{epsilon,if,while,for,return,type,{,id}
IfStmt	{if}	{epsilon,if,while,for,return,type,{,id}
ElseStmt	{else,epsilon}	{epsilon,if,while,for,return,type,{,id}
OtherStmt	{while,for,return,type,{,id}	{epsilon,if,while,for,return,type,{,id}
ExprStmt	{id}	{epsilon,if,while,for,return,type,{,id}
Expr	{-,id,num,{}	{;,),,,}
EqualExpr	{-,id,num,{}	{;,),==,,}
AddExpr	{-,id,num,{}	{;,),==,+,,,}
MulExpr	{-,id,num,{}	{;,),==,+,*,,,}
UnaryExpr	{-,id,num,{}	{;,),==,+,*,,,}
OtherExpr	{id,num,{}	{;,),==,+,*,,,}
ArgList	{epsilon,-,id,num,{}	{}

IfStmt -> if (Expr) then Block · ElseStmt

여기까지 읽은 상태에서

ElseStmt -> · else Block (shift)

ElseStmt -> · epsilon (reduce)

Shift/reduce 충돌이 발생한다

이때 Follow(IfStmt) = Follow(ElseStmt) = { epsilon, if, while, for, return, type, {, id }

Lookahead가 else일때, else는 Follow(ElseStmt)에 들어있지 않으므로 reduce(epsilon) 이 아닌 shift(else) 한다. 만약 lookahead가 Follow(ElseStmt)에 속하는 다른 토큰이라면 reduce 한다.

이러한 방법으로 if (Expr) then Block ElseStmt 형태의 상황에서 else를 shift 해서 else 구문까지 처리하고, if (Expr) then Block의 else가 없는 상황에서는 reduce(epsilon) 되어서 else 없는 if구문을 처리한다.

3. Parser Implementation

3-1. 구현

```
from typing import List, Tuple, Union
from collections import defaultdict
import re

class ParseTree:
    def __init__(self, symbol, children=None, value=None):
        self.symbol = symbol
        self.children = children if children is not None else []
        self.value = value

    def __str__(self):
        return self._to_string()

    def _to_string(self, level=0):
        indent = " " * level
        result = f"{indent}{self.symbol}"
```



```

        stack.append(terminal)
        stack.append(int(action[1:]))
        index += 1

    elif action[0] == "r": # reduce 경우
        rule_index = int(action[1:])
        lhs, rhs = given_grammar[rule_index]

        children = []
        if rhs == ["epsilon"]:
            pass
        else:
            for _ in range(len(rhs)):
                if len(stack) < 1: # 상태 pop
                    return False, ErrorReport(
                        index,
                        f"reduce 위한 상태 pop 이전에 스택 언더플로우 발생. Rule:
{lhs} -> {' '.join(rhs)}",
                    )
                stack.pop()

                if len(stack) < 1: # 노드 pop
                    return False, ErrorReport(
                        index,
                        f"reduce 위한 노드 pop 이전에 스택 언더플로우 발생. Rule:
{lhs} -> {' '.join(rhs)}",
                    )
                node_to_add = stack.pop() # ParseTree 노드 pop

                if not isinstance(node_to_add, ParseTree):
                    return False, ErrorReport(
                        index,
                        f"스택의 parseTree 위치에 Type 이 존재함, got
{type(node_to_add)}. Rule: {lhs} -> {' '.join(rhs)}",
                    )
                children.insert(0, node_to_add)

        parent_node = ParseTree(lhs, children)

        # goto
        prev_state = stack[-1]
        if not isinstance(prev_state, int):
            return False, ErrorReport(
                index,
                f"Invalid state {prev_state}, 축소후 스택 top 에 int 가 아닌 값이
있음",
            )

        goto = SLR_TABLE.get(prev_state, {}).get(lhs)
        if goto is None:

```

```

        return False, ErrorReport(
            index, f"상태 {prev_state} 에서 심볼 {lhs}에 대한 goto 가 정의되어
있지 않습니다."
        )

    if not isinstance(goto, int):
        return False, ErrorReport(
            index, f"현재 goto 값 {goto}는 정수가 아님."
        )

    stack.append(parent_node)
    stack.append(goto)

    return False, ErrorReport(index, "SLR parser loop 비정상 종료")

def make_table(raw_grammar):
    #  $\epsilon$  처리 & 인덱스 매핑
    grammar = []
    for lhs, rhs in raw_grammar:
        grammar.append((lhs, [] if rhs == ["epsilon"] else rhs))

    nonterminals = {lhs for lhs, _ in grammar}
    terminals = set(
        sym for _, rhs in grammar for sym in rhs
        if sym not in nonterminals
    ) | {"$"}

    # FIRST
    FIRST = {X: set() for X in nonterminals}
    for t in terminals:
        FIRST[t] = {t}
    FIRST["epsilon"] = {"epsilon"}

    changed = True
    while changed:
        changed = False
        for A, alpha in grammar:
            if not alpha:
                if "epsilon" not in FIRST[A]:
                    FIRST[A].add("epsilon"); changed = True
            else:
                for X in alpha:
                    before = len(FIRST[A])
                    FIRST[A] |= (FIRST[X] - {"epsilon"})
                    if "epsilon" in FIRST[X]:
                        continue
                    break
                else:
                    if "epsilon" not in FIRST[A]:

```

```

        FIRST[A].add("epsilon"); changed = True

# FOLLOW
FOLLOW = {A: set() for A in nonterminals}
FOLLOW["Program"].add("$")
changed = True
while changed:
    changed = False
    for A, alpha in grammar:
        trailer = FOLLOW[A].copy()
        for X in reversed(alpha):
            if X in nonterminals:
                before = len(FOLLOW[X])
                FOLLOW[X] |= trailer
                if len(FOLLOW[X]) != before:
                    changed = True
                if "epsilon" in FIRST[X]:
                    trailer |= (FIRST[X] - {"epsilon"})
                else:
                    trailer = FIRST[X] - {"epsilon"}
            else:
                trailer = {X}

# LR(0) 아이템·closure·goto 정의
def closure(items):
    C = set(items)
    while True:
        new = set()
        for A, alpha, dot in C:
            if dot < len(alpha):
                B = alpha[dot]
                if B in nonterminals:
                    for lhs, rhs in grammar:
                        if lhs == B:
                            new.add((B, tuple(rhs), 0))
        if new <= C:
            break
        C |= new
    return C

def goto(I, X):
    moved = {(A, alpha, dot+1) for A, alpha, dot in I if dot < len(alpha) and
alpha[dot] == X}
    return closure(moved)

# 상태 생성
states = []
init = closure({"Program", tuple(grammar[0][1]), 0})
states.append(init)
changed = True

```

```

while changed:
    changed = False
    for I in states:
        for X in nonterminals | terminals:
            J = goto(I, X)
            if J and J not in states:
                states.append(J)
                changed = True

# 상태별 인덱스
state_ids = {frozenset(s): i for i, s in enumerate(states)}

# ACTION / GOTO 테이블 생성
action_table = {}
goto_table = {}

prod_index = { (lhs, tuple(rhs)): i for i, (lhs, rhs) in enumerate(grammar) }

for I in states:
    i = state_ids[frozenset(I)]
    for A, alpha, dot in I:
        # shift
        if dot < len(alpha):
            a = alpha[dot]
            if a in terminals:
                j = state_ids[frozenset(goto(I, a))]
                action_table[(i, a)] = ('shift', j)
            else:
                # accept
                if A == "Program":
                    action_table[(i, '$')] = ('accept', None)
                else:
                    # reduce
                    idx = prod_index[(A, alpha)]
                    for a in FOLLOW[A]:
                        action_table[(i, a)] = ('reduce', idx)

# GOTO 채우기
for A in nonterminals:
    J = goto(I, A)
    if J:
        j = state_ids[frozenset(J)]
        goto_table[(i, A)] = j

SLR_TABLE = {} # Dict[int, Dict[str, Union[str,int]]]

# 1) ACTION 항목: shift/reduce/accept
for (state, term), (kind, val) in action_table.items():
    entry = None
    if kind == 'shift':
        entry = f"s{val}"

```

```

        elif kind == 'reduce':
            entry = f"r{val}"
        elif kind == 'accept':
            entry = "acc"
        SLR_TABLE.setdefault(state, {})[term] = entry

# 2) GOTO 항목: next state 번호 (int)
for (state, nonterm), dest in goto_table.items():
    SLR_TABLE.setdefault(state, {})[nonterm] = dest

return SLR_TABLE

```

```

given_grammar = [
    ("Program", ["Program"]),
    ("Program", ["DeclList"]),
    ("DeclList", ["Decl", "DeclList"]),
    ("DeclList", ["epsilon"]),
    ("Decl", ["VarDecl"]),
    ("Decl", ["FuncDecl"]),
    ("VarDecl", ["type", "id", ";"]),
    ("VarDecl", ["type", "id", "=", "Expr", ";"]),
    ("FuncDecl", ["type", "id", "(", "ParamList", ")", "Block"]),
    ("ParamList", ["Param", ",", "ParamList"]),
    ("ParamList", ["Param"]),
    ("ParamList", ["epsilon"]),
    ("Param", ["type", "id"]),
    ("Block", [{"", "StmtList", "}"]),
    ("StmtList", ["Stmt", "StmtList"]),
    ("StmtList", ["epsilon"]),
    ("Stmt", ["IfStmt"]),
    ("Stmt", ["OtherStmt"]),
    ("IfStmt", ["if", "(", "Expr", ")", "then", "Block", "ElseStmt"]),
    ("ElseStmt", ["else", "Block"]),
    ("ElseStmt", ["epsilon"]),
    ("OtherStmt", ["while", "(", "Expr", ")", "Stmt"]),
    ("OtherStmt", ["for", "(", "Expr", ";", "Expr", ";", "Expr", ")", "Stmt"]),
    ("OtherStmt", ["return", "Expr", ";"]),
    ("OtherStmt", ["VarDecl"]),
    ("OtherStmt", ["ExprStmt"]),
    ("OtherStmt", ["Block"]),
    ("ExprStmt", ["id", "=", "Expr", ";"]),
    ("Expr", ["EqualExpr"]),
    ("EqualExpr", ["EqualExpr", "==", "AddExpr"]),
    ("EqualExpr", ["AddExpr"]),
    ("AddExpr", ["AddExpr", "+", "MulExpr"]),
    ("AddExpr", ["MulExpr"]),
    ("MulExpr", ["MulExpr", "*", "UnaryExpr"]),
    ("MulExpr", ["UnaryExpr"]),
    ("UnaryExpr", ["-", "UnaryExpr"]),
    ("UnaryExpr", ["OtherExpr"]),

```

```

("OtherExpr", ["id", "(", "ArgList", ")"]),
("OtherExpr", ["id"]),
("OtherExpr", ["num"]),
("OtherExpr", ["(", "Expr", ")"]),
("ArgList", ["Expr", ",", "ArgList"]),
("ArgList", ["Expr"]),
("ArgList", ["epsilon"]),
]

def preprocess_read_tokens(data):
    # 합침
    content = ' '.join(data) if isinstance(data, list) else data

    # '==' 을 우선 한 덩어리로
    content = re.sub(r'==', ' == ', content)

    # 실제 문법에서 쓰는 기호들 띄어쓰기
    # '=' 은 단일 등호만 아래 정규식으로 처리
    for ch in ['+', '*', '-', '(', ')', '{', '}', ',', ';']:
        content = content.replace(ch, f' {ch} ')

    # '=' 토큰 분리
    content = re.sub(r'(?<![=])=(?![=])', ' = ', content)

    return [tok for tok in content.split() if tok]

```

3-2. 결과 예시

예시 1 :

```

type id ( type id , type id ) {
    type id = id ( id , num ) ;
    while ( id == num ) {
        id = id + num ;
    }
    return id ;
}

```

}

결과 :

Accepted

Program

DeclList

Decl

FuncDecl

type

id

(

ParamList

Param

type

id

,

ParamList

Param

type

id

)

Block

{

StmtList

Stmt

OtherStmt

VarDecl

type

id

=

Expr

EqualExpr

AddExpr

MulExpr

UnaryExpr

OtherExpr

id

(

ArgList

Expr

EqualExpr

AddExpr

MulExpr

UnaryExpr

OtherExpr

id

,

ArgList

Expr

EqualExpr

AddExpr

MulExpr

UnaryExpr

OtherExpr

num

)

.

StmtList

Stmt

OtherStmt

while

(

Expr

EqualExpr

EqualExpr

AddExpr

MulExpr

UnaryExpr

OtherExpr

id

$$\frac{1}{2} \frac{d}{dt} \left(\frac{1}{2} \frac{d}{dt} \right)$$

AddExpr

MulExpr

UnaryExpr

OtherExpr

num

)

Stmt

OtherStmt

Block

{

StmtList

Stmt

OtherStmt

ExprStmt

id

=

Expr

EqualExpr

AddExpr

AddExpr

MulExpr

UnaryExpr

OtherExpr

id

+

MulExpr

UnaryExpr

OtherExpr

num

;

StmtList

}

StmtList

Stmt

OtherStmt

return

Expr

EqualExpr

AddExpr

MulExpr

UnaryExpr

OtherExpr

id

;

StmtList

}

DeclList

예제 2:

```
type id () {  
    if ( num == num ) then {  
        id = id * id + (-id);  
    } else {  
        type id ;  
    }  
}
```

결과 :

Accepted

Program

DeclList

Decl

FuncDecl

type

id

(

ParamList

)

Block

{

StmtList

Stmt

IfStmt

if

(

Expr

EqualExpr

EqualExpr

AddExpr

MulExpr

UnaryExpr

OtherExpr

num

==

AddExpr

MulExpr

UnaryExpr

OtherExpr

num

)

then

Block

{

StmtList

Stmt

OtherStmt

ExprStmt

id

=

Expr

EqualExpr

AddExpr

AddExpr

MulExpr

MulExpr

UnaryExpr

OtherExpr

id

*

UnaryExpr

OtherExpr

id

+

MulExpr

UnaryExpr

OtherExpr

(

Expr

EqualExpr

AddExpr

MulExpr

UnaryExpr

-

UnaryExpr

OtherExpr

id

)

```
                ;
            StmtList
        }
ElseStmt
    else
    Block
    {
        StmtList
        Stmt
        OtherStmt
        VarDecl
        type
        id
        ;
        StmtList
    }
    StmtList
}
DeclList
```

예제 3 : (if문에서 Block이 아닌 경우)

```
type id () {  
    if ( num == num ) then  
        id = id * id + (-id);  
    else  
        type id ;  
}
```

결과 :

Error at token 12: unexpected token 'id', expected one of ['{', 'Block']

예제 4 : (토큰이 아닌 문자)

```
type id ( ) {  
    id = id & num;  
}
```

결과 :

Error at token 8: unexpected token '&', expected one of ['(', ')', ';', '::', '*', '+', '==']

예제 5 : (콤마 빼기)

```
type id(type id type id) {  
    return id;  
}
```

결과 :

Error at token 5: unexpected token 'type', expected one of [',', ')']